Recomputing an AR bundle file from scratch

the geometry, with nothing skipped

What this document is for

A bundle file (`bundles/room.xml`) tells `ar_track_alvar` where several printed markers sit *relative to each other*. If the markers get moved, the file is wrong, and every pose that depends on it is wrong in a way that still looks plausible.

The file cannot be measured conveniently with a tape measure (the numbers are corner positions in a frame that may not physically exist), but it *can* be computed from two observations. This document derives that computation from first principles, so that it can be redone after any rearrangement without rediscovering the algebra.

Nothing here is harder than vectors and matrix multiplication. The difficulty is entirely bookkeeping: which frame is a number expressed in, and which direction does a transform go. Sections 1–3.2 are about exactly that, and they are the part worth reading slowly.

Contents

1 Points, frames, and what a "pose" is **1**

2 Rotation matrices: what the columns mean **1**

3 Homogeneous transforms: rotation and translation in one object **2**
- 3.1 Composition: the cancellation rule . . . 2
- 3.2 The inverse . . . 3

4 What is actually in the bundle file **3**

5 From four corners to a transform **4**

6 From a transform back to four corners **5**

7 What the camera gives us **5**

8 The derivation **5**

9 Turning the published numbers into matrices **6**
- 9.1 Averaging many samples . . . 6

10 Worked example with the real numbers **7**
- 10.1 Input A — master seen through marker 11 . . . 7
- 10.2 Input B — master seen through marker 12 . . . 7
- 10.3 Input X — the old XML entry for marker 12 . . . 8
- 10.4 Step 1: invert A . . . 8
- 10.5 Step 2: the discrepancy $A^{-1}B$ . . . 8
- 10.6 Step 3: apply it . . . 9
- 10.7 Step 4: back to corners . . . 9

11 Why the master frame is a free choice **9**

12 What this method cannot tell you **9**

13 The recipe, next time the room changes **10**

1 Points, frames, and what a "pose" is

A *point* in the room is a physical thing: a corner of a printed square. It has no numbers of its own. Numbers appear only once a *frame* is chosen.

A frame is an origin plus three perpendicular unit axes $\hat{\mathbf{x}}, \hat{\mathbf{y}}, \hat{\mathbf{z}}$, forming a right-handed set ($\hat{\mathbf{x}} \times \hat{\mathbf{y}} = \hat{\mathbf{z}}$). The coordinates of a point in that frame are the three distances you travel along those axes to reach it.

The same physical point therefore has different coordinates in different frames. Writing $\mathbf{p}$ with no indication of frame is the single most common source of error in this kind of work, so in this document every quantity carries its frame.

Four frames appear:

C	the camera (`radius_infra1_optical_frame`)
M	the *master* frame of the bundle (marker index 0; in our room, fictional)
11	the frame of the printed marker with id 11
12	the frame of the printed marker with id 12

A *pose* of frame B relative to frame A is two things together: where B's origin sits, expressed in A coordinates, and how B's axes are turned relative to A's. That is exactly the information needed to convert coordinates from B into A, which is why "pose of B in A" and "transform from B to A" are the same object.

2 Rotation matrices: what the columns mean

A rotation is stored as a 3×3 matrix $R_{A\leftarrow B}$. The definition worth memorising is this one:

The columns of $R_{A\leftarrow B}$ are the axes of frame B, written in A coordinates.

So if B's $\hat{\mathbf{x}}$ axis points along A's $\hat{\mathbf{z}}$ axis, the first column of $R_{A\leftarrow B}$ is $(0, 0, 1)^\top$. That is the whole content of the matrix; everything else follows.

Why that definition makes the matrix do the right thing: a point with B-coordinates $\mathbf{p}_B = (p_1, p_2, p_3)^\top$ is, by the meaning of coordinates, the combination $p_1\hat{\mathbf{x}}_B + p_2\hat{\mathbf{y}}_B + p_3\hat{\mathbf{z}}_B$. Writing each of those axis vectors in A coordinates and adding them up is precisely the matrix–vector product:

$$\mathbf{p}_A \;=\; p_1\,(\mathrm{col}_1) + p_2\,(\mathrm{col}_2) + p_3\,(\mathrm{col}_3) \;=\; R_{A\leftarrow B}\,\mathbf{p}_B. \tag{1}$$

Properties, and how to check a matrix is really a rotation. Because the columns are perpendicular unit vectors:

$$R_{A\leftarrow B}^\top R_{A\leftarrow B} = I \qquad \text{(columns orthonormal)} \tag{2}$$

$$\det R_{A\leftarrow B} = +1 \qquad \text{(right-handed, not mirrored)} \tag{3}$$

$$R_{A\leftarrow B}^{-1} = R_{A\leftarrow B}^\top \qquad \text{(the inverse is free)} \tag{4}$$

The determinant check matters. A matrix with $\det = -1$ is a reflection: it will look almost right, it will pass the orthonormality test, and it will produce a world that is mirror-imaged. If a fit ever returns $\det = -1$, something upstream is wrong — do not "fix" it by flipping a sign at random.

How big is a rotation? The angle θ of a rotation matrix R is

$$\theta = \arccos\left(\frac{\operatorname{trace}(R) - 1}{2}\right). \tag{5}$$

This is useful for sanity checks: "how far did my correction turn the marker?" is answered by applying (5) to the correction matrix. Clamp the argument into $[-1, 1]$ before calling `arccos`, because rounding will occasionally hand you 1.0000000002.

3 Homogeneous transforms: rotation and translation in one object

Rotation alone cannot express "and also, the origin is over there". The full transform is

$$\mathbf{p}_A = R_{A\leftarrow B}\,\mathbf{p}_B + \mathbf{t}_{A\leftarrow B}, \tag{6}$$

where $\mathbf{t}_{A\leftarrow B}$ is the position of B's origin in A coordinates.

Carrying a matrix and a vector around separately is error-prone, so both are packed into one 4×4 matrix:

$$T_{A\leftarrow B} = \begin{pmatrix} R_{A\leftarrow B} & \mathbf{t}_{A\leftarrow B} \\ \mathbf{0}^\top & 1 \end{pmatrix}, \qquad \begin{pmatrix} \mathbf{p}_A \\ 1 \end{pmatrix} = T_{A\leftarrow B} \begin{pmatrix} \mathbf{p}_B \\ 1 \end{pmatrix}. \tag{7}$$

The trailing 1 on the point is what lets the translation column act; the bottom row $(0, 0, 0, 1)$ keeps that 1 in place after multiplication. There is no deeper meaning — it is a bookkeeping trick that makes composition work.

3.1 Composition: the cancellation rule

The reason the arrow notation is worth the extra typing:

$$\boxed{T_{A\leftarrow B}\,T_{B\leftarrow C} = T_{A\leftarrow C}} \tag{8}$$

The inner labels must match and they cancel, exactly like units. Read it right to left: start with C-coordinates, the right-hand factor turns them into B-coordinates, the left-hand factor turns those into A-coordinates.

If a product you have written does not cancel — say $T_{A\leftarrow B}\,T_{C\leftarrow B}$ — it is meaningless, and no amount of transposing will rescue it. One of the two factors needs inverting first. Checking cancellation before computing anything catches most mistakes in this subject.

3.2 The inverse

Reversing a transform is not "put a minus sign on everything". Starting from $\mathbf{p}_A = R\,\mathbf{p}_B + \mathbf{t}$ and solving for $\mathbf{p}_B$:

$$\mathbf{p}_A - \mathbf{t} = R\,\mathbf{p}_B \tag{9}$$

$$R^\top(\mathbf{p}_A - \mathbf{t}) = \mathbf{p}_B \qquad (\text{using } R^{-1} = R^\top) \tag{10}$$

$$\mathbf{p}_B = R^\top \mathbf{p}_A \;+\; \big(-R^\top \mathbf{t}\big). \tag{11}$$

So the inverse has rotation $R^\top$ and translation $-R^\top\mathbf{t}$:

$$T^{-1}_{A\leftarrow B} \;=\; T_{B\leftarrow A} \;=\; \begin{pmatrix} R^\top & -R^\top\mathbf{t} \\ \mathbf{0}^\top & 1 \end{pmatrix}. \tag{12}$$

The $-R^\top\mathbf{t}$ term is the one people drop. Geometrically: the new translation has to be the old origin's position *seen from the other frame*, which involves rotating it, not just negating it.

4 What is actually in the bundle file

Here is the relevant part of the file, before the correction:

```
<marker index="0" status="1">
  <corner x="6.5"  y="-6.5"    z="0" />
  <corner x="6.5"  y="6.5"     z="0" />
  <corner x="-6.5" y="6.5"     z="0" />
  <corner x="-6.5" y="-6.5"    z="0" />
</marker>
<marker index="12" status="1">
  <corner x="-59"  y="-4"  z="51" />
  <corner x="-59"  y="9"   z="51" />
  <corner x="-59"  y="9"   z="64" />
  <corner x="-59"  y="-4"  z="64" />
</marker>
```

Three facts, none of which the file states:

1. **The units are centimetres.** The markers are 13 cm squares and the half-width 6.5 appears everywhere. Everything published on ROS topics is in metres. Convert once, early, and write down which unit you are in.
2. **All corner coordinates are expressed in the master frame M.** The master is the marker with `index="0"`. This is why its own corners are the clean symmetric numbers $(\pm 6.5, \pm 6.5, 0)$: a marker expressed in its own frame sits at the origin, flat in the $z = 0$ plane.
3. **The master need not exist physically.** In our room, marker 0 was never printed or placed. That is allowed — M is just the coordinate system the other markers are described in. It has a consequence we return to in Section 11.

The local corner convention. Reading marker 0's entry tells us the order and orientation alvar expects for a marker's own corners. With half-width $s = 6.5$ cm:

$$\mathbf{l}_0 = \begin{pmatrix} s \\ -s \\ 0 \end{pmatrix}, \quad \mathbf{l}_1 = \begin{pmatrix} s \\ s \\ 0 \end{pmatrix}, \quad \mathbf{l}_2 = \begin{pmatrix} -s \\ s \\ 0 \end{pmatrix}, \quad \mathbf{l}_3 = \begin{pmatrix} -s \\ -s \\ 0 \end{pmatrix}. \tag{13}$$

So in a marker's own frame: $\hat{\mathbf{x}}$ runs from corner 3 to corner 0, $\hat{\mathbf{y}}$ runs from corner 0 to corner 1, and $\hat{\mathbf{z}}$ points out of the printed face. *The order of the four corners in the XML is part of the data.* Permuting them silently rotates or flips the marker.

5 From four corners to a transform

Given a marker's four corners $\mathbf{w}_0 \dots \mathbf{w}_3$ in master coordinates, we want $T_{M \leftarrow k}$, the pose of that marker in the master frame.

Use the definition from Section 2 — the columns of a rotation matrix are the child frame's axes expressed in the parent. We know from (13) which pairs of corners define those axes, so we just read them off:

$$\hat{\mathbf{x}} = \frac{\mathbf{w}_0 - \mathbf{w}_3}{\|\mathbf{w}_0 - \mathbf{w}_3\|} \qquad \text{(the local } +x \text{ direction)} \tag{14}$$

$$\hat{\mathbf{y}} = \frac{\mathbf{w}_1 - \mathbf{w}_0}{\|\mathbf{w}_1 - \mathbf{w}_0\|} \qquad \text{(the local } +y \text{ direction)} \tag{15}$$

$$\hat{\mathbf{z}} = \hat{\mathbf{x}} \times \hat{\mathbf{y}} \qquad \text{(out of the face, right-handed)} \tag{16}$$

and

$$R_{M\leftarrow k} = \big(\,\hat{\mathbf{x}} \;\big|\; \hat{\mathbf{y}} \;\big|\; \hat{\mathbf{z}}\,\big), \qquad \mathbf{t}_{M\leftarrow k} = \tfrac{1}{4}\textstyle\sum_{i=0}^{3} \mathbf{w}_i. \tag{17}$$

The translation is just the centroid of the four corners, because the local corners in (13) are symmetric about the origin and therefore average to zero.

Why $\hat{\mathbf{z}}$ comes from a cross product rather than the corners. The four corners are coplanar, so they contain no information about which side of the plane is "front" beyond the handedness convention. Taking $\hat{\mathbf{x}} \times \hat{\mathbf{y}}$ enforces $\det = +1$ by construction.

Checks worth doing every time.

- $\|\mathbf{w}_0 - \mathbf{w}_3\|$ and $\|\mathbf{w}_1 - \mathbf{w}_0\|$ should both equal the marker width (13 cm). If they differ, the XML entry is not a square and was hand-fitted.
- $\hat{\mathbf{x}} \cdot \hat{\mathbf{y}}$ should be 0. If it is not, the entry is not even a rectangle.
- Applying (17) to marker 0's own corners must give the identity. It is a free unit test of your implementation, and it confirms you have the corner order right.

When the corners are not perfectly consistent. Marker 11's entry in our file has side lengths 13.0 and 12.9 cm, so no rigid transform reproduces it exactly. Equations (14)–(16) will still return something, but it quietly favours whichever two corners you happened to use. The honest alternative is the *Kabsch* fit, which uses all four corners and minimises the total squared error:

1. Centre both sets: $\tilde{\mathbf{l}}_i = \mathbf{l}_i - \bar{\mathbf{l}}$ and $\tilde{\mathbf{w}}_i = \mathbf{w}_i - \bar{\mathbf{w}}$.
2. Build $H = \sum_i \tilde{\mathbf{l}}_i \tilde{\mathbf{w}}_i^{\top}$ (a 3×3 matrix).
3. Take the singular value decomposition $H = USV^{\top}$.
4. Set $R = V\,\mathrm{diag}(1, 1, d)U^{\top}$ with $d = \mathrm{sign}\big(\det(VU^{\top})\big)$.
5. Then $\mathbf{t} = \bar{\mathbf{w}} - R\bar{\mathbf{l}}$.

The d term is what prevents the answer from coming out as a reflection. The leftover per-corner distances after applying $R, \mathbf{t}$ are a genuine residual: for a clean entry they are 0, and for marker 11 they are 0.5 mm at every corner.

6 From a transform back to four corners

To write a marker back into the XML, run Section 5 backwards. For each of the four local corners in (13):

$$\mathbf{w}_i \;=\; R_{M\leftarrow k}\,\mathbf{l}_i \;+\; \mathbf{t}_{M\leftarrow k}, \qquad i = 0, 1, 2, 3, \tag{18}$$

keeping the same order, and converting metres back to centimetres.

Do not expect the new numbers to be as tidy as the old ones. The original entry had every x equal to -59 because the marker was assumed perfectly parallel to a wall. A measured marker is not, so all four x values will differ slightly. That is the measurement, not a mistake.

7 What the camera gives us

`ar_track_alvar` publishes visualisation markers on `visualization_marker`. Two namespaces matter:

`basic_shapes`, id $= k$	where alvar thinks *the individual marker k* is
`main_shapes`, id $= 0$	where alvar thinks *the bundle's master frame* is

Both are poses in the camera frame C. The second one is the interesting one, and the reason is worth stating carefully:

> To report the master pose, alvar must combine what it *sees* (a marker) with what it has been *told* (that marker's position in the bundle file). So `main_shapes` is part measurement, part assumption. **If the bundle file is wrong, `main_shapes` is wrong — and it is wrong by a different amount depending on which marker is currently visible.** That difference is the signal we exploit.

Hence the experiment:

1. Cover marker 12. Only 11 is visible. Record `main_shapes`. Call it A.
2. Cover marker 11. Only 12 is visible. Record `main_shapes`. Call it B.
3. **Do not move the camera between the two recordings.**

If the bundle file were correct, A and B would be identical. They are not.

8 The derivation

Name the quantities, each with its frames:

symbol	meaning	known?
$A = T^{(11)}_{C\leftarrow M}$	master pose, computed by alvar from marker 11	measured
$B = T^{(12)}_{C\leftarrow M}$	master pose, computed by alvar from marker 12	measured
$X = T_{M\leftarrow 12}$	marker 12's pose in the master frame, *per the old XML*	known, wrong
$C_{12} = T_{C\leftarrow 12}$	where marker 12 truly is, seen by the camera	unknown
X_{true}	what the XML *should* say	what we want

Step 1: write down what alvar did to produce B. It measured marker 12 directly, giving $T_{C\leftarrow 12}$, then chained through the bundle file to reach the master:

$$B = T_{C\leftarrow M} = \underbrace{T_{C\leftarrow 12}}_{\text{measured}}\ \underbrace{T_{12\leftarrow M}}_{\text{from XML}} = C_{12}\,X^{-1}. \tag{19}$$

Check the cancellation: $C \leftarrow 12$ then $12 \leftarrow M$ gives $C \leftarrow M$. Good. Note $T_{12\leftarrow M} = X^{-1}$, because the XML stores $X = T_{M\leftarrow 12}$, the other direction.

Step 2: eliminate the unknown. Rearranging (19) gives the camera's actual view of marker 12:

$$C_{12} = B\,X. \tag{20}$$

This is the key move. C_{12} is a *measurement* that was never published directly, but it can be reconstructed, because alvar's error came entirely from the XML and we know what the XML said.

Step 3: decide what the master frame means. The correct XML entry is marker 12's pose in the master frame:

$$X_{\text{true}} = T_{M\leftarrow 12} = T_{M\leftarrow C}\,T_{C\leftarrow 12} = A^{-1}\,C_{12}, \tag{21}$$

where we took A — the estimate from marker 11 — as the definition of $T_{C\leftarrow M}$. Section 11 explains why that is a legitimate choice and not an assumption being smuggled in.

Step 4: substitute (20) and finish.

$$\boxed{X_{\text{true}} \;=\; A^{-1}\,B\,X} \tag{22}$$

Read the result right to left: take the old XML entry, apply the discrepancy $A^{-1}B$ between the two master estimates, and you have the corrected entry. The factor $A^{-1}B$ is *exactly* the error that was in the file — it is the transform that moves "where 12 claimed the master was" onto "where 11 says the master is".

Sanity check on the formula. If the XML had been right, A and B would be equal, so $A^{-1}B = I$ and $X_{\text{true}} = X$: the formula leaves a correct file alone. That check costs nothing and catches an inverted factor immediately.

9 Turning the published numbers into matrices

ROS publishes orientations as quaternions (q_x, q_y, q_z, q_w) — note that ROS puts w *last*, while most textbooks put it first. Getting this wrong produces a rotation that is wrong but not obviously wrong.

First normalise, $\|q\| = 1$, then

$$R = \begin{pmatrix} 1-2(q_y^2+q_z^2) & 2(q_xq_y - q_zq_w) & 2(q_xq_z + q_yq_w) \\ 2(q_xq_y + q_zq_w) & 1-2(q_x^2+q_z^2) & 2(q_yq_z - q_xq_w) \\ 2(q_xq_z - q_yq_w) & 2(q_yq_z + q_xq_w) & 1-2(q_x^2+q_y^2) \end{pmatrix}. \tag{23}$$

Always verify $R^\top R = I$ and $\det R = +1$ afterwards. If the quaternion was not normalised, the first check fails and tells you so.

9.1 Averaging many samples

A single frame is noisy. Record a few seconds and average.

Translations average normally: add them up, divide by the count.

Rotations do not. Averaging rotation matrices entry by entry gives a matrix that is not a rotation. There is also a sign trap: q and $-q$ describe the *same* rotation, so a naive mean of samples that happen to differ in sign tends toward zero.

The standard fix:

1. Normalise every quaternion.
2. Pick one as a reference. For every other sample, if $q \cdot q_{\text{ref}} < 0$, replace q by $-q$. Now they all live in the same hemisphere.
3. Stack them as the rows of an $N \times 4$ matrix Q.
4. Form $Q^\top Q$ (a 4×4 symmetric matrix) and take its eigenvector for the largest eigenvalue. That eigenvector is the average quaternion.

Step 4 looks like magic but is just least squares: it finds the unit quaternion most aligned with all the samples at once. For tightly clustered samples it agrees with the naive mean to several decimals; it is the sign handling in step 2 that does most of the work in practice.

Report the spread, not just the mean. The standard deviation of the translations and the maximum angle between each sample and the mean tell you whether the recording was any good. For the data this document is based on, the 58 samples of B had a translation spread of 0.3 mm and a rotation spread of 0.03° — i.e. the camera and markers really were static.

Note what that does and does not prove: it is *precision*, not accuracy. A systematically wrong pose can be extremely repeatable.

10 Worked example with the real numbers

Units are metres throughout this section; conversion back to centimetres happens at the very end.

10.1 Input A — master seen through marker 11

One recorded sample:

$$\mathbf{t}_A = \begin{pmatrix} 0.181993 \\ 0.633671 \\ -0.018187 \end{pmatrix}, \qquad q_A = (0.236931,\ 0.694806,\ -0.650365,\ 0.195279).$$

Through (23):

$$R_A = \begin{pmatrix} -0.811460 & 0.583247 & -0.036821 \\ 0.075237 & 0.041779 & -0.996290 \\ -0.579545 & -0.811220 & -0.077783 \end{pmatrix}.$$

10.2 Input B — master seen through marker 12

Average of 58 samples:

$$\mathbf{t}_B = \begin{pmatrix} -0.213286 \\ 0.584412 \\ 0.262373 \end{pmatrix}, \qquad q_B = (0.223091,\ 0.678418,\ -0.676997,\ 0.177917),$$

$$R_B = \begin{pmatrix} -0.837152 & 0.543597 & -0.060659 \\ 0.061799 & -0.016188 & -0.997957 \\ -0.543468 & -0.839190 & -0.020042 \end{pmatrix}.$$

10.3 Input X — the old XML entry for marker 12

Corners $(-59, -4, 51)$, $(-59, 9, 51)$, $(-59, 9, 64)$, $(-59, -4, 64)$ in cm, i.e. $(-0.59, -0.04, 0.51)$ etc. in metres. Applying (14)–(17):

$$\begin{aligned}
\hat{\mathbf{x}} &= \frac{(-0.59, -0.04, 0.51) - (-0.59, -0.04, 0.64)}{0.13} = \frac{(0, 0, -0.13)}{0.13} = (0, 0, -1), \\
\hat{\mathbf{y}} &= \frac{(-0.59, 0.09, 0.51) - (-0.59, -0.04, 0.51)}{0.13} = \frac{(0, 0.13, 0)}{0.13} = (0, 1, 0), \\
\hat{\mathbf{z}} &= \hat{\mathbf{x}} \times \hat{\mathbf{y}} = (0, 0, -1) \times (0, 1, 0) = (1, 0, 0),
\end{aligned}$$

so, stacking those as *columns*,

$$X = \begin{pmatrix} 0 & 0 & 1 & -0.590 \\ 0 & 1 & 0 & 0.025 \\ -1 & 0 & 0 & 0.575 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

The centroid $(-0.590,\ 0.025,\ 0.575)$ is the average of the four corners, as promised.

It is worth pausing on what this matrix says: marker 12's own x axis points along the master's $-z$, its y along the master's $+y$, and its face normal along the master's $+x$. In words, it is on a wall to one side, facing inward — which matches the physical room, and is a good sign that the corner order was read correctly.

10.4 Step 1: invert A

Using (12), $R_A^\top$ and $-R_A^\top \mathbf{t}_A$:

$$A^{-1} = \begin{pmatrix} -0.811460 & 0.075237 & -0.579545 & 0.089465 \\ 0.583247 & 0.041779 & -0.811220 & -0.147374 \\ -0.036821 & -0.996290 & -0.077783 & 0.636607 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Observe that the rotation block is R_A transposed — rows and columns swapped — while the translation is *not* $-\mathbf{t}_A$. That is (12) in action.

10.5 Step 2: the discrepancy $A^{-1}B$

$$A^{-1}B = \begin{pmatrix} 0.998929 & 0.044023 & -0.014245 & 0.154450 \\ -0.044812 & 0.997143 & -0.060814 & -0.460199 \\ 0.011527 & 0.061388 & 0.998047 & 0.041808 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

This single matrix is the entire error that was sitting in the bundle file. Decoded:

- translation $(15.4,\ -46.0,\ 4.2)$ cm — the two estimates of the master frame disagreed by about half a metre;
- rotation angle, from (5) with trace $= 2.994119$: $\theta = 4.39°$.

A near-identity rotation block with a large translation is the signature of "a marker got moved", as opposed to "a marker got re-oriented". Worth noting for diagnosis next time.

10.6 Step 3: apply it

$$X_{\text{true}} = (A^{-1}B)\,X = \begin{pmatrix} 0.014245 & 0.044023 & 0.998929 & -0.442008 \\ 0.060814 & 0.997143 & -0.044812 & -0.443800 \\ -0.998047 & 0.061388 & 0.011527 & 0.610419 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Centre moved from $(-59.0,\ 2.5,\ 57.5)$ to $(-44.2,\ -44.4,\ 61.0)$ cm, a shift of 49.3 cm, with the marker turned by 4.39°.

10.7 Step 4: back to corners

Via (18) with $s = 0.065$ m, then $\times 100$:

```
<marker index="12" status="1">
  <corner x="-44.39" y="-50.47" z="54.16" />
  <corner x="-43.82" y="-37.50" z="54.95" />
  <corner x="-44.01" y="-38.29" z="67.93" />
  <corner x="-44.58" y="-51.26" z="67.13" />
</marker>
```

The four x values are no longer identical, and the z values no longer come in two matching pairs. That is the 4.39° showing up in the numbers.

11 Why the master frame is a free choice

In Step 3 of Section 8 we declared that A — the estimate through marker 11 — *defines* the master frame. Why is that allowed, when A is just as much a measurement as B?

Because the master frame is not observable. Marker 0 was never placed. Nothing in the room, in the camera, or in the data knows where M "really" is. What the bundle file genuinely encodes is the *relative* geometry of markers 11 and 12; the master is only the coordinate system that relationship is written in.

Shifting M anywhere changes every entry in the file, but changes no prediction the system makes — the camera poses that come out the other end are identical. A degree of freedom with no observable consequence is called a *gauge* freedom, and the usual practice is to pin it to something convenient. Leaving marker 11's entry untouched is convenient: it means a re-measurement only ever edits the markers that moved.

Two consequences:

- Marker 11's entry has become *definitional*. It cannot be "wrong" any more in the sense that matters, because it is what M means. (It can still be wrong in the sense of not being a 13 cm square — see Section 5.)
- If you ever want M to coincide with something physical — the floor, say, as was originally intended — that is a separate decision requiring a separate measurement, and it changes every entry in the file at once.

12 What this method cannot tell you

There is no residual. Two pose observations, one rigid transform to solve for, six unknowns and six constraints: the problem is *exactly determined*. It fits perfectly, always, by construction. Applying the correction makes A and B agree because that is literally the equation we solved — not because the answer is right.

So this procedure cannot report its own accuracy. To get a residual you need redundancy: more viewpoints than unknowns. Record the pair from three or four camera positions and solve for the one X that best fits all of them; the leftover disagreement is then a real error estimate.

The output inherits the weaker input. The result is only as good as the worse of A and B. In the example, B averaged 58 samples while A was a single frame — so the final numbers carry single-frame noise regardless of how carefully B was measured. Record both with the same care.

Single-marker pose has an ambiguity. Estimating a pose from one planar square is ill-conditioned out of plane: there are two poses that project to nearly the same image, differing by a flip about an axis in the marker plane. Solvers sometimes pick the wrong one, especially for markers that are small in the image, viewed close to head-on, or far away. A sudden tens-of-degrees change between consecutive samples is the signature. Both of A and B are single-marker estimates, so this is the failure mode to watch for.

A camera that moved invalidates everything. The derivation assumes A and B share one camera frame C. Nothing in the data reveals a bump between recordings. Take both recordings back to back.

13 The recipe, next time the room changes

1. Point the camera so that *both* markers are comfortably in view, reasonably large in the image, and not edge-on. Then do not touch it.

2. Cover marker 12. Record `visualization_marker` for a few seconds. Keep only the messages with `ns: "main_shapes"`. This is A.

3. Uncover 12, cover 11. Record again, same way. This is B.

4. Average each set: mean translation, eigenvector-method quaternion. Print the spreads and look at them before continuing.

5. Build A and B as 4×4 matrices using (23).

6. Read the *old* XML entry for marker 12 and build X with (14)–(17). Convert cm to m. Check the side lengths are 13 cm and that marker 0's entry gives the identity.

7. Compute $X_{\text{true}} = A^{-1}BX$ using (12) for the inverse.

8. Report the shift and the rotation angle. A shift of the size you remember moving the marker is the single best confirmation available.

9. Generate the new corners with (18), convert back to cm, write them into the file keeping the corner order.

10. **Verify independently**: run with both markers visible and watch `main_shapes` while covering one, then the other. If the master pose does not jump, the relative geometry is right. This is the only check that is not circular.

If there are more than two markers. Repeat steps 2–9 once per extra marker, always against the same reference A, and always leaving the reference marker's entry alone. With n markers you need n recordings, one per marker, all from the same camera pose.